# MARK SCHEME · TEACHER COPY

12BR · Topic 19: Computational Thinking and Problem-solving

## Cambridge A Level Computer Science 9618

Paper 3  Advanced Theory

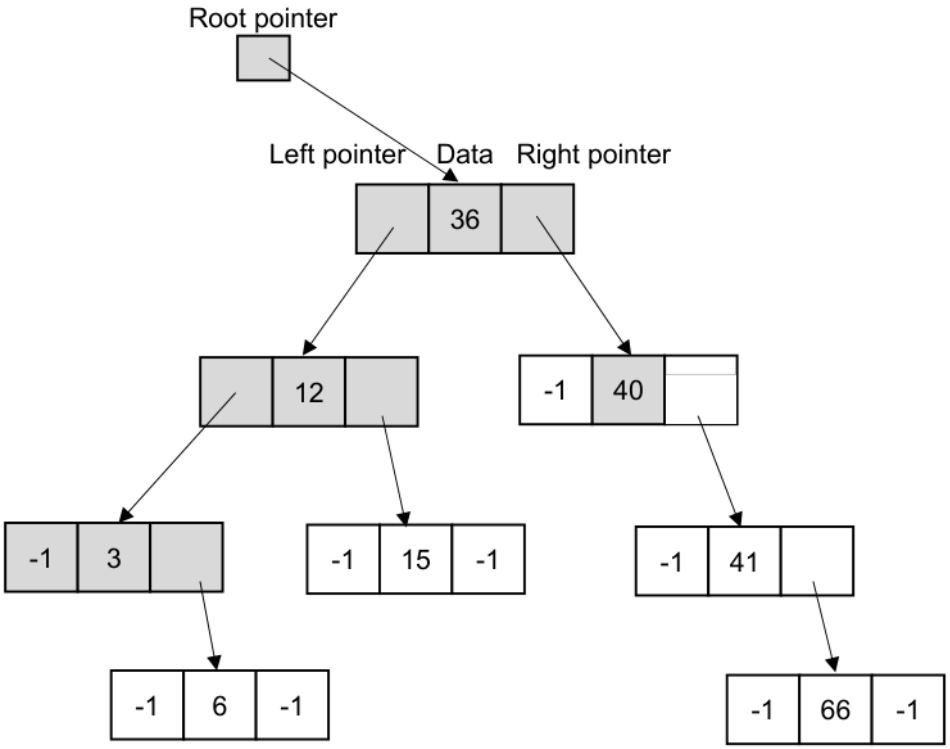
## Total marks: 46

Official Cambridge mark scheme extracts for each question in the matching exam paper, with question numbers changed to match the exam. Source of each question is shown above its rows.

For teacher / admin use only. Do not give to students before they have attempted the paper.

Mark scheme extracts © Cambridge University Press & Assessment.

## Question 1 · Oct/Nov 2025 Paper 31, original Q4 (a),(b),(c) [7 marks]

| Question | Answer | Marks |
|---|---|---|
| 1(a) | **One** mark per mark point<br>MP1 Any two nodes added correctly with correct data (6, 15, 41, 66) and arrows<br>MP2 Remaining two nodes added with correct data (6, 15, 41, 66) and all nodes with connecting arrows starting from pointer boxes<br>MP3 Correct null pointers (-1) added throughout<br>MP4 … with no entries in other pointer boxes and all nodes correctly positioned and connected<br>  | 4 |
| 1(b) | **One** mark per mark point<br>MP1 A technique used to solve problems using a function/procedure/subroutine that calls itself (general case)<br>MP2 … until the terminating condition / base case is achieved, when no further recursive calls are made | 2 |

| | | |
|---|---|---|
| 1(c) | **One** mark<br>• Stack | **1** |

## Question 2 · May/June 2026 Paper 33, original Q2 [4 marks]

| Question | Answer | Marks | Guidance |
|---|---|---|---|
| 2 | Examiners must award **whole marks** (not half marks, or other fractions). | | |
| 2 | **One** mark for each correct word in the correct place (**Max 4**)<br><br>protocol<br>layer<br>adjacent<br>Application<br><br>The implementation of a **protocol** suite can be viewed as a stack. Each **layer** of the stack has its own specific functionality and can only communicate with **adjacent** layers of the stack. User interaction is associated with the highest level of the stack, which is the **Application** layer. | **4** | |

| | | | | | |
|---|---|---|---|---|---|
| 2 | W | N1 | 6 | 15 | 21 |

## Question 3 · May/June 2026 Paper 33, original Q10 (a),(b) [9 marks]

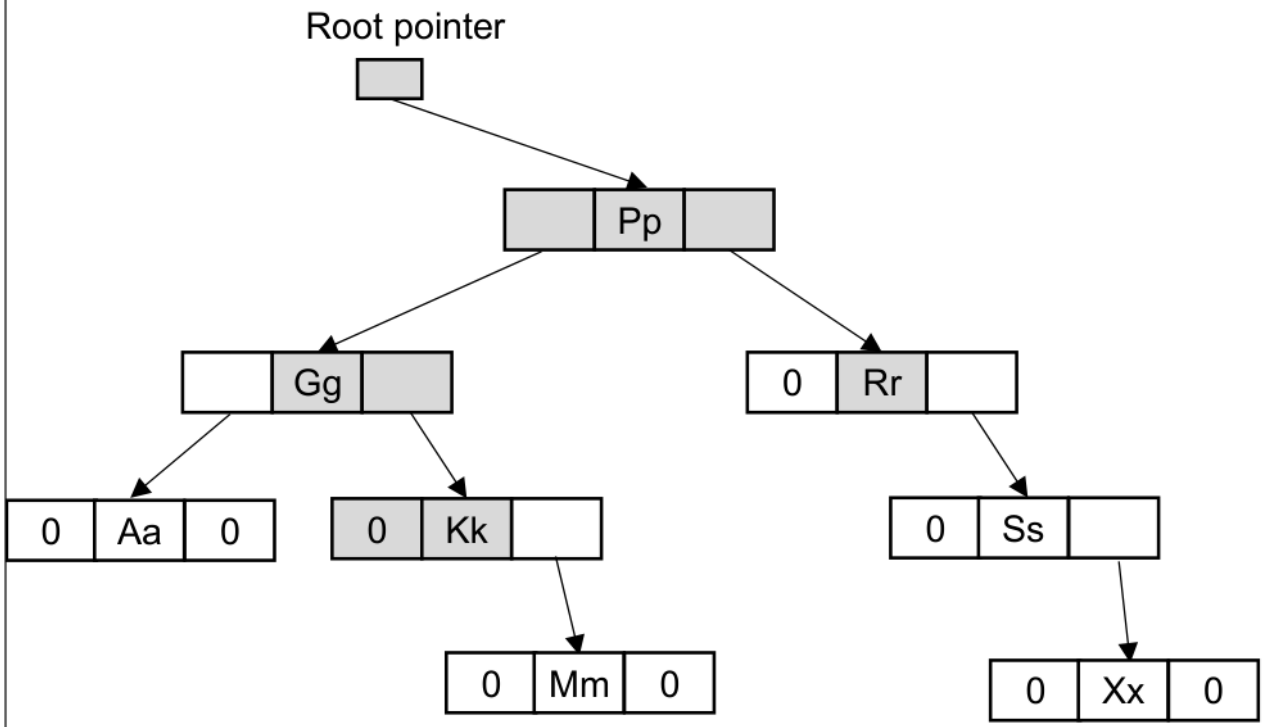
| Question | Answer | Marks | Guidance |
|---|---|---|---|
| 3(a)(i) | **One** mark for every two correct rows (**Max 2**) | **2** | Don't accept mid as description for the centre index, but middle is OK |

| Identifier | Data type | Description |
|---|---|---|
| Left | INTEGER | Lowest index in the search area |
| Right | INTEGER | Highest index in the search area |
| Mid | INTEGER | Centre/middle index in the search area |
| Target | STRING | The item to be found |

| | | | |
|---|---|---|---|
| 3(a)(ii) | **One** mark per mark point (**Max 5**)<br>MP1 Function header correctly completed<br>MP2 Left and right indexes correctly initialised<br>MP3 Appropriate loop **and** escape condition.<br>MP4 Setting of the search index (Mid)<br>MP5 Conditional statement to find item<br>MP6 Conditional statement **and** changing of left and right indexes if item not found<br>MP7 Return of location if item/target found **and** -1 if not<br><br>Example completed algorithm<br><br>`FUNCTION PartSearch(Target : STRING) RETURNS INTEGER`<br>`  Left ← 1`<br>`  Right ← 1000000`<br>`  WHILE Left <= Right`<br>`  Mid ← (Left + Right) DIV 2)` **OR**<br>`  Mid ← DIV(Left + Right, 2)`<br>`  IF PartName[Mid] = Target THEN`<br>`  RETURN Mid`<br>`  ELSE`<br>`  IF PartName[Mid] < Target THEN`<br>`  Left ← Mid + 1`<br>`  ELSE`<br>`  Right ← Mid - 1`<br>`  ENDIF`<br>`  ENDIF`<br>`  ENDWHILE`<br>`  RETURN -1`<br>`ENDFUNCTION` | **5** | All required identifiers (variables. Indexes and array) are given in the question, so must be used.<br><br>Grey shaded areas given in question.<br><br>Watch out for incorrect `RETURN` / `RETURNS` in candidate responses.<br><br>Accept == for = |
| 3(b) | **One** mark per mark point (**Max 2**)<br>MP1 The time it takes to execute a binary search increases logarithmically as the number of search items increases // The time increases linearly as the number of search items increases exponentially<br>MP2 Binary search worst case Big O is O(log n)/ O($\log_2$ n) | **2** | The Big O notation must be completely correct (including brackets). |

**Question 4 · Oct/Nov 2025 Paper 32, original Q11 (a),(b) [6 marks]**

| Question | Answer | Marks |
|---|---|---|

| | | |
|---|---|---|
| 4(a) | **One** mark per mark point<br>MP1 Any two nodes added correctly with correct data (Aa, Mm, Ss, Xx)<br>MP2 Remaining two nodes added with correct data (Aa, Mm, Ss, Xx) and all nodes with connecting arrows starting from correct pointer boxes<br>MP3 Correct null pointers (0) added throughout<br>MP4 … with no entries in other pointer boxes and all nodes correctly positioned and connected.<br>  <br> | **4** |

| | | |
|---|---|---|
| 4(b) | **One** mark for feature and **one** mark for example (**Max 2**)<br><br>• Recursion is beneficial for algorithms when a problem naturally breaks down into smaller versions of itself.<br>• … such as calculating a factorial / mathematical series / Fibonacci / compound interest / tree traversal / evaluation of RPN expressions. | **2** |

**Question 5 · May/June 2026 Paper 31, original Q10 (a),(b),(c) [9 marks]**

| Question | Answer | Marks | Guidance |
|---|---|---|---|
| 5(a) | **One** mark per mark point (**Max 5**)<br>MP1 Correct outer loop iterating through entire array starting at second element<br>MP2 Temporary variables both initialised to current value of arrays<br>MP3 Insertion variable initialised<br>MP4 Correct inner loop criteria<br>MP5 Elements of both arrays inserted as appropriate<br>MP6 Insertion variable decremented<br>MP7 Temporary values replaced in correct place in arrays<br><br>Sample algorithm<br><br>`FOR Count ← 1 to 499`<br>`  Temp1 ← MemberID[Count]`<br>`  Temp2 ← Names[Count]`<br>`  Insert ← Count - 1`<br>`  WHILE Insert >= 0 AND MemberID[Insert] > Temp1`<br>`  MemberID[Insert + 1] ← MemberID[Insert]`<br>`  Names[Insert + 1] ← Names[Insert]`<br>`  Insert ← Insert - 1`<br>`  ENDWHILE`<br>`  MemberID[Insert + 1] ← Temp1`<br>`  Names[Insert + 1] ← Temp2`<br>`NEXT Count` | **5** | MP2 must be `Temp1` and `Temp2`<br>MP3 must be `Insert`<br><br>Do not accept bubble sort answers |

| | | | |
|---|---|---|---|
| 5(b) | **One** mark per mark point (**Max 3**)<br>MP1 Array size // Number of elements in the array<br>MP2 … time efficiency will improve if the number of elements to be sorted are fewer<br>MP3 Initial order<br>MP4 … if the array is partially sorted the insertion sort works better | **3** | Allow the opposite for MP2 (worse/more).<br><br>Allow the opposite for MP4 (reversed/slower). |
| 5(c) | $O(n^2)$ // $O(N^2)$ | **1** | Must be completely correct (including brackets). |

**Question 6 · Oct/Nov 2025 Paper 33, original Q12 (a),(b),(c) [7 marks]**

| Question | Answer | Marks |
|---|---|---|
| 6(a) | **One** mark for any **two** correct ADTs (**Max 1**)<br>• Binary tree<br>• Graph<br>• Linked list<br>• Queue<br>• Stack | **1** |

| | | |
|---|---|---|
| 6(b) | **One** mark for each marking point (**Max 4**)<br>MP1 Temporary assignment of the element being 'inserted' before inner loop<br>MP2 Appropriate inner loop<br>MP3 Check if current `DataArray` content is > `Value`<br>MP4 Moving data to adjacent element as required<br>MP5 Appropriate updating of `Position` variable<br>MP6 Re-insertion of the element outside the inner loop<br><br>**Example algorithm**<br><br>`FOR Index ← 2 to 1000`<br>`  Value ← DataArray[Index]`<br>`  Position ← Index - 1`<br>`  IF DataArray[Position] > Value THEN`<br>`  WHILE Position >= 1 AND DataArray[Position] > Value`<br>`  DataArray[Position + 1] ← DataArray[Position]`<br>`  Position ← Position - 1`<br>`  ENDWHILE`<br>`  DataArray[Position + 1] ← Value`<br>`  ENDIF`<br>`NEXT Index` | **4** |
| 6(c) | **One** mark for each marking point<br>MP1 The performance of a sorting routine should improve if the data is already partially sorted // The performance of a sorting routine is likely to be worse if the data is completely out of order<br>MP2 The sort may take longer if the number of items to be sorted is larger // The sort may take less time if the number of items to be sorted is fewer. | **2** |

## Question 7 · Oct/Nov 2025 Paper 33, original Q13 [4 marks]

| Question | Answer | Marks |
|---|---|---|
| 7 | **One** mark for each marking point<br>MP1 Correct `Index` and `Target` columns<br>MP2 Correct `Numbers[5]` column<br>MP3 Correct `Numbers[6]` and `Numbers[7]` columns<br>MP4 Correct `Numbers[8]` column and no incorrect data added to any other of columns `[1, 2, 3, 4, 9, 10]` | **4** |

| | | Numbers | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Index** | **Target** | **[1]** | **[2]** | **[3]** | **[4]** | **[5]** | **[6]** | **[7]** | **[8]** | **[9]** | **[10]** |